



Diplomado en Data Science Aplicada con Python para la Toma de Decisiones

Clase 36: RAG multimodal — por qué tu PDF rompe el RAG simple

Carlos Enrique Mosquera Trujillo

9 de Junio, 2026

Dónde quedamos en la clase 35



Armamos el **chatbot Arca de Ética y Cumplimiento**: indexamos un PDF de 8 páginas (el código de ética público), corrimos PyPDF, chunkeamos, embebimos con mpnet y buscamos por co-seno. Funcionó. Citaba el documento y rechazaba los 3 ataques de injection.

Funcionó porque el PDF era texto plano, sin figuras complicadas. Hoy le tiramos un PDF real.

La pregunta del día

En clase 35 armamos un chatbot Arca que cita el Código de Ética. Hoy queremos uno que lea el Reporte Anual Integrado 2025 de Arca Continental: 189 páginas, 415 figuras, gráficos de sostenibilidad, tablas financieras. ¿Por qué el RAG simple se rompe aquí, qué le ponemos para que funcione, y cuánto código real escribimos — vs. cuánto le pedimos a una librería profesional?

Acompaña los 6 bloques. La respondemos al final, frame 49.

Bloque o

RAG: qué es,
para qué sirve,
y cómo le tapa la boca al modelo que inventa

¿Qué es RAG, en una frase?

Retrieval-Augmented Generation: antes de responder, el modelo recupera los documentos relevantes y los pega en su propio contexto.



Idea grande: *primero buscar, después hablar.*

¿Para qué sirve RAG — 3 casos típicos en Arca



Consultar políticas

“¿cómo reporto algo anónimo?”
respuesta cita el manual



Analizar reportes

“¿qué metas de
sostenibilidad para 2030?”



Atender clientes

FAQ + tickets pasados
sin alucinar

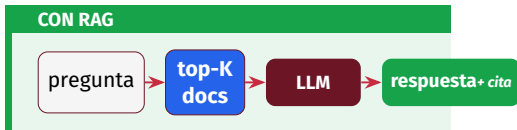
El factor común: **el modelo solo, sin contexto, inventa**. Le damos los documentos **a la hora de responder** y el problema desaparece.

Cómo combate la alucinación



El modelo genera “el código de ética dice...” sin haber visto el código de ética.

Puede acertar de casualidad. O inventar.



El modelo lee **el chunk literal** antes de contestar.

Si la respuesta no está en el contexto, la instrucción es “*decí no aparece*”.

No es magia: el modelo *sigue pudiendo* inventar. Pero la tasa cae fuerte cuando le pones el documento delante.

🏠 ¿Qué me llevo? — Bloque 0

- ▶ **RAG = recuperar antes de generar.** Esa es la idea grande.
- ▶ Sirve cuando el modelo *solo* no tiene esa info confiable (políticas internas, reportes propios, FAQ).
- ▶ **Es la defensa más fuerte contra alucinación**, junto con `temperature=0` y un buen system prompt.

RAG = recuperar antes de generar. Eso es lo que mata la alucinación. → Bloque 1: el RAG simple que ya armaste en clase 35.

Bloque 1

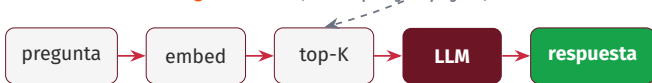
RAG simple
(esto ya lo armaste en clase 35)

El pipeline de clase 35 — 3 pasos

1. INDEXAR *(se hace una vez)*



2. RECUPERAR + 3. GENERAR *(cada vez que alguien pregunta)*



Tres pasos. **Indexar** se hace una vez. **Recuperar** y **generar** se hacen *cada vez* que alguien pregunta.

El chatbot Ética que sí funcionó

```
texto = "\n".join(p.extract_text() for p in
    ↳ PdfReader("etica.pdf").pages)
chunks = [c for c in texto.split("\n\n") if len(c) > 80]

embs = embedder.encode(chunks, normalize_embeddings=True)
def rag(q, k=3):
    qe = embedder.encode([q], normalize_embeddings=True)[0]
    top = np.argsort(-(embs @ qe))[:k]
    return llm(SYSTEM, "CONTEXTO:\n" + "\n---\n".join(chunks[i]
    ↳ for i in top) +
        f"\n\nPREGUNTA: {q}")
```

¿Por qué funcionó? El PDF de Ética era texto plano y los chunks 'split("\n\n")' tenían sentido. Si el PDF tiene figuras, columnas, tablas... esa última línea se cae.

¿Qué me llevo? — Bloque 1

- ▶ El **pipeline 3 pasos** (indexar / recuperar / generar) sigue siendo el mismo.
- ▶ Lo que cambia hoy: **el primer paso**. Cómo entra el PDF a tu sistema.
- ▶ Todo lo demás (embedder, vector store, LLM, retrieval) lo dejamos tal cual.

Tres pasos. Texto plano, esto ya lo haces. → Bloque 2: ahora le tiramos un PDF real y vemos cómo se rompe.

Bloque 2

Cuando el PDF
rompe el RAG simple

Un PDF no es un .txt

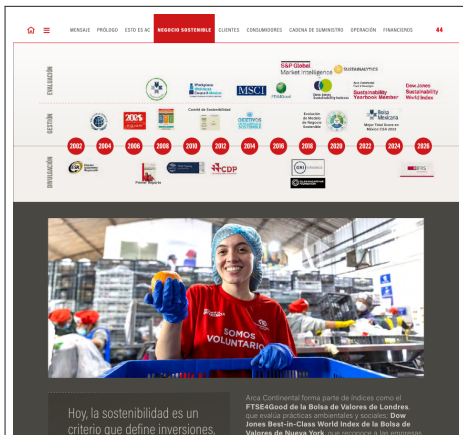
Encabezados / pie
Texto principal
Tablas
Figuras (PNG / vectoriales)
Layout (columnas, posiciones)

PyPDF
→

Un PDF guarda **varias capas** encima de la página. PyPDF lee la capa de texto y la concatena *como cadena lineal*, perdiendo el contexto visual.

- ▶ Las **figuras** no las ve.
- ▶ Las **tablas** las aplanan en sopa de números.
- ▶ Las **columnas** las mezcla.

El paciente: página 44 del Reporte Arca 2025



Una página típica de un reporte corporativo real:

- ▶ Encabezado con **menú** de secciones.
- ▶ Una **barra** con **22 logos** de organismos (FTSE4Good, S&P, MSCI...).
- ▶ Una **foto** grande de una trabajadora.
- ▶ **Tres columnas** de texto.
- ▶ Un **recuadro destacado** con el título de la sección.

Véámosla primero. Después la pasamos por PyPDF.

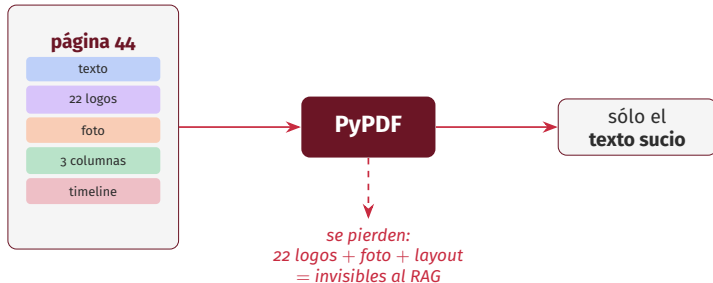
Lo que PyPDF saca de esa página (output real)

```
PdfReader("reporte_arca.pdf").pages[43].extract_text()
```

```
DIVULGACIÓN GESTIÓN EVALUACIÓN
Comité de Sostenibilidad
Evolución de Modelo de Negocio
S
ostenible
Hoy, la sostenibilidad es un
criterio que define inversiones,
decisiones y relaciones con
los grupos de interés, validado
por estándares globales
Arca Continental forma parte de índices como el
FTSE4Good de la Bolsa de Valores de Londres,
que evalúa prácticas
Dow Jones Best
.
in
.
Class World Index de la Bolsa de
```

⚠️ "S \n ostensible", "Dow Jones Best \n · \n in \n · \n Class". Las palabras se rompen. Y todo viene en una columna mezclada.

Y las 22 figuras de esa página... ¿dónde están?



Si alguien te pregunta “¿en qué índices ESG está Arca?” y la respuesta solo está en los logos visuales... tu RAG no lo encuentra.

Y las tablas terminan así...

código	equipo	acción
ERR-001	Compresor	Verificar rodamientos.
ERR-002	Llenadora	Limpiar filtro entrada.
ERR-003	Etiquetado	Ajustar rodillo aplicador.

Cómo se ve en el PDF

PyPDF

```
"código equipo acción ERR-001 Compresor
Verificar rodamientos. ERR-002
Llenadora
Limpiar filtro entrada. ERR-003
Etiquetador
Ajustar rodillo aplicador."
```

Los **tres campos de cada fila** quedan pegados con el header y con las filas siguientes. El RAG ve *una sopa*, no una tabla.

Si preguntas "¿qué hago con ERR-002?", el chunk recuperado puede traer la acción de ERR-003 mezclada.

¿Por qué pasa esto?

Un PDF está pensado para que un *humano* mire la página y entienda dónde está cada cosa. **No para que una máquina lo lea linealmente.**

PyPDF te entrega el texto en *el orden en el que el PDF lo guarda internamente* — que muchas veces es por bloques, columnas o fragmentos sueltos sin saber a cuál pertenece cada uno.

- ▶ **Las figuras** son un objeto aparte. PyPDF las salta.
- ▶ **Las tablas** no tienen “modo tabla” — son rectangulitos con texto.
- ▶ **El layout** (columnas, recuadros, márgenes) PyPDF no lo respeta.

Resumen: PyPDF no lee mal. Es que el PDF *no es un .txt* y necesitamos más herramientas.

🏠 ¿Qué me llevo? — Bloque 2

- ▶ Tu **chatbot de clase 35 SIRVE** cuando el PDF es texto plano (ej. el código de ética).
- ▶ Sobre un **reporte real con figuras, tablas, columnas** — el parser solo de PyPDF no alcanza.
- ▶ Las **figuras** se pierden completamente. Las **tablas** se convierten en sopa. El **layout** se mezcla.

*Un PDF real es más complejo de lo que PyPDF puede leer. → Bloque 3: para entender lo que el PDF Sí tiene (figuras), necesitamos **modelos multimodales**.*

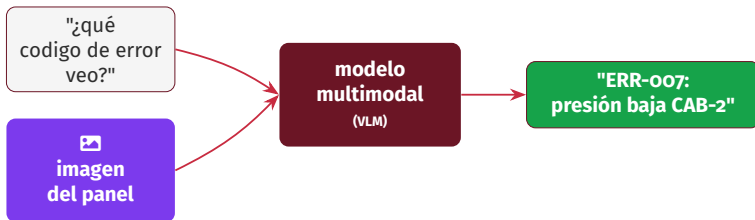
Bloque 3

Modelos multimodales
(el modelo también puede ver)

¿Qué quiere decir “multimodal”?

Hasta ahora nuestros modelos recibían **una sola modalidad**: texto.

Un *modelo multimodal* acepta **texto + imagen** (o audio, o video) **en la misma llamada**, y la respuesta es texto.



El modelo que vamos a usar — Llama 4 Scout 17B

Por qué este

Mismo proveedor	Groq, el que ya conocemos de clase 34-35.
Cliente idéntico	El mismo <code>client.chat.completions.create()</code> de la API OpenAI.
Free tier	Suficiente para indexar un reporte completo con cuidado.
Acepta base64	Vos le pasas la imagen como <code>data:image/png;base64,...</code>
Reemplazo	Llama 3.2 Vision ya no está en Groq; Scout es el sucesor.

Nada de stack nuevo. Los mismos 5 imports. Solo cambia `model="..."` y el formato del mensaje (lista con texto + imagen).

Anatomía del request multimodal

```
import base64
with open("figura.png", "rb") as f:
    b64 = base64.b64encode(f.read()).decode()

r = client.chat.completions.create(
    model="meta-llama/llama-4-scout-17b-16e-instruct",
    messages=[{"role": "user", "content": [
        {"type": "text", "text": "Describe esta imagen en
        ↪ 2 frases."},
        {"type": "image_url", "image_url": {"url":
        ↪ f"data:image/png;base64,{b64}"}}],
    ]}],
    max_tokens=200, temperature=0,
)
print(r.choices[0].message.content)
```

Lo único *nuevo*: content ahora es una **lista** con dos partes (texto + image_url) en vez de un str.

Demo en vivo — una foto del reporte + una pregunta



figura extraída del reporte (pag. 52 aprox.)

Prompt

“Describe esta foto en 2 frases. ¿Quién aparece? ¿Qué está haciendo?”

Respuesta REAL de Scout

“En la imagen se ve a una mujer sonriente con un gorro de Coca Cola. La mujer está apoyando sus brazos sobre una carga de botellas de Coca Cola que transporta en un carrito.”

Reconoce el gorro, la marca, las botellas, el carrito. Eso

🏠 ¿Qué me llevo? — Bloque 3

- ▶ **Multimodal** no es ciencia rara. Es “el modelo también puede ver”.
- ▶ En Groq: una sola línea cambia. Mismo cliente, mismo patrón.
- ▶ Lo que recibe el modelo sigue siendo *prompt* + *contexto*. Solo que ahora el contexto puede incluir *una imagen*.

Multimodal = el modelo no solo lee, también ve. → Bloque 4: y cuando le pedimos que describa una imagen con palabras, eso se llama captioning.

Bloque 4

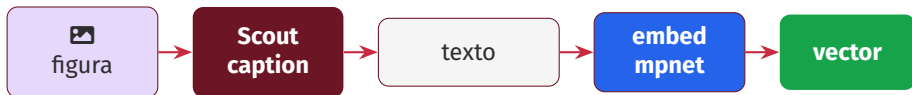
Image captioning
(la imagen se vuelve texto)

¿Qué es *image captioning*?



Le pides al VLM una sola cosa: *“describeme esta imagen en X frases”*. Sale **texto**. Y eso es lo que metes tu RAG al vector store.

Por qué resuelve el problema



- ▶ La imagen entra como texto al vector store.
- ▶ **Tu RAG de clase 35 SIGUE FUNCIONANDO** sin cambios.
- ▶ Cuando alguien pregunta “¿qué muestra el panel de la línea 2?”, el chunk recuperado puede ser *el caption de una foto*.

No cambiamos la arquitectura, solo le agregamos una fuente más de texto.

Demo — captions reales sobre el Reporte Arca 2025



pag 70

"Dos hombres disfrutando comida en un mercado. Se ven las marcas Inca Kola y Coca-Cola."



pag 47

"Fila de camiones de Coca-Cola en un almacén, listos para la distribución de productos."



pag 61

"Persona realizando pago móvil en una tienda de conveniencia, con refrigerador marca yompi!"

Scout identificó marcas, escenas, tecnología y contexto comercial. Tres llamadas, tres textos listos para indexar.

🏠 ¿Qué me llevo? — Bloque 4

- ▶ **Captioning** convierte cualquier imagen en *texto descriptivo*.
- ▶ El texto entra al RAG *como si fuera otro chunk del manual*.
- ▶ Costo: **una llamada al VLM por imagen al indexar**. Cero costo extra en query.

Captioning convierte la imagen en texto. Y ahí tu RAG ya funciona. → Bloque 5: ahora, ¿hay alguna librería que ya haga todo esto por nosotros?

Bloque 5

La librería que ya hace todo:
`pymupdf4llm`

pymupdf4llm: parser de PDFs pensado para RAG

Una librería Python que toma tu PDF y te devuelve **markdown limpio**, **detecta tablas** como markdown tabular, y **guarda las figuras como PNG** con un nombre que las ata a su página.

- ▶ Python puro, `pip install pymupdf4llm`. Sin servidor, sin API key.
- ▶ Reusa pymupdf (el mismo que mostramos antes).
- ▶ Licencia AGPL (open source).
- ▶ **Velocidad real medida:** 31 páginas del reporte Arca → **6.8 s** en una laptop. Sale 67 KB de markdown + 108 PNGs.

Qué hace — una sola línea

```
import pymupdf4llm

md = pymupdf4llm.to_markdown(
    "reporte_arca_2025.pdf",
    write_images=True,                # guarda las figuras
    image_path="figs/",
    image_format="png",
)
print(len(md), "chars de markdown")  # 66612
print(os.listdir("figs/")[:3])       #
↳ ['...pdf-0001-03.png', ...]
```

Una llamada. Te devuelve el texto en *markdown* (con encabezados, bullets, negritas detectadas) y te extrae cada imagen del PDF como un PNG aparte.

Lo que pymupdf4llm saca de esa misma página 44

```
pymupdf4llm.to_markdown(...)
```

```
MENSAJE PRÓLOGO      ESTO ES AC      NEGOCIO SOSTENIBLE      CLIENTES
```

```
**44**
```

```
***=> picture [612 x 761] intentionally omitted <===**
```

Comité de Sostenibilidad

Evolución de Modelo de Negocio Sostenible

Hoy, la sostenibilidad es un criterio que define inversiones, decisiones y relaciones con los grupos de interés, validado por estándares globales que reconocen la gestión responsable y la creación de valor a largo plazo.

Arca Continental forma parte de índices como el FTSE4Good de la Bolsa de Valores de Londres, que evalúa prácticas ambientales y sociales; Dow Jones Best-in-Class World Index...

Las palabras llegan enteras, los encabezados marcados con ****...****. Los lugares donde había figura quedan marcados con `<== picture ==>` — la imagen está en disco con su *filename*.

El diff de la página 44 — PyPDF vs pymupdf4llm

PyPDF

Comité
de
Sostenibilidad
Evolución
de
Modelo
de
S
ostenible
Dow Jones Best
.
in
.
Class

[figuras: ninguna]

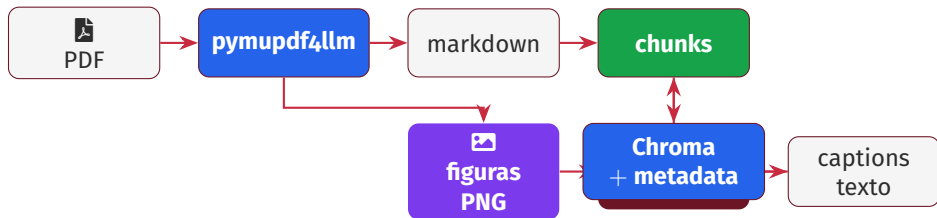
pymupdf4llm

Comité de Sostenibilidad
Evolución de Modelo de Negocio
Sostenible
Dow Jones Best-in-Class World
Index...

[figuras extraídas en figs/:
pdf-0004-02.png
pdf-0004-03.png
pdf-0004-05.png]

✓ Texto limpio + figuras en archivos. Lo que falta: *describir las figuras*. Eso es captioning.

El pipeline completo — pymupdf4llm + captioning



Texto + figuras descritas, todo *como texto*, todo en el mismo store, cada chunk con `metadata={..., modalidad: texto|figura}`.

Si pymupdf4llm te queda chico — el mapa

Herramienta	Qué agrega	Costo / fricción
pymupdf4llm	markdown limpio, figuras a PNG	free, Python, 7 s / 31 pp
Docling (IBM)	layout, OCR, tablas estructuradas	free, pesado: ~8 min / página en este PDF
LlamaParse	layout vía LLM, tablas excelentes	API paga (free tier limitado)
Unstructured	ingesta de muchos formatos, layout	free, dependencias pesadas
RAGFlow (app)	<i>pipeline completo con UI web</i>	Docker, multiusuario

Default Arca: **pymupdf4llm**. Si el PDF tiene *tablas críticas para el negocio* y los números no pueden salir mal: LlamaParse o Docling. Si quieres algo *con UI para que los usuarios suban sus propios PDFs*: RAGFlow.

🏠 ¿Qué me llevo? — Bloque 5

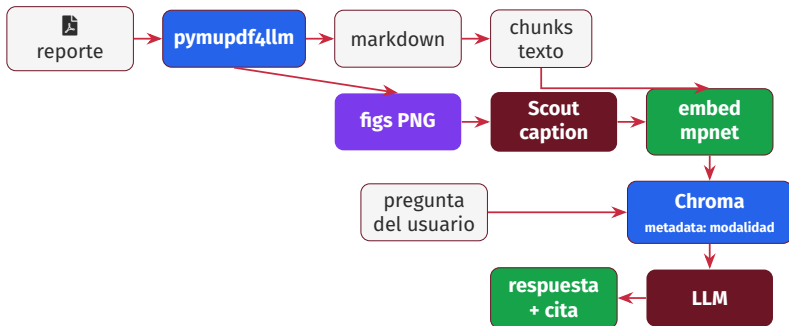
- ▶ Probablemente **no quieres escribir tu propio parser**. Hay librerías que ya hacen el 90 %.
- ▶ Para arrancar: **pymupdf4llm**. Una línea. Te da markdown + figuras.
- ▶ Cuando el negocio depende de **precisión sobre tablas** o tienes **PDFs escaneados**, subes a Docling / LlamaParse.

Esto ya existe como librería. Docling te lo resuelve. → Bloque 6: armemos el asistente Arca v2.

Bloque 6

El asistente Arca v2
sobre el Reporte 2025

Arquitectura del asistente v2



Texto + captions van al mismo store. La metadata (modalidad) deja filtrar en el query.

Pipeline en código — una pasada

```
md = pymupdf4llm.to_markdown("reporte.pdf",  
                             write_images=True,  
                             ↪ image_path="figs/")  
chunks_txt = sliding_window(md, tam=400)  
chunks_img = [caption(p) for p in glob("figs/*.png")]  
  
docs    = chunks_txt + chunks_img  
metas   = [{"modalidad": "texto"}]*len(chunks_txt) + \  
          [{"modalidad": "figura"}]*len(chunks_img)  
  
embs = embedder.encode(docs,  
    ↪ normalize_embeddings=True).tolist()  
col.add(documents=docs, embeddings=embs, metadatas=metas,  
        ids=[f"d_{i}" for i in range(len(docs))])
```

El rag(pregunta) de clase 35 no cambia ni una línea.

Query #1 — pregunta de texto sobre el reporte

Pregunta

“¿En qué índices de sostenibilidad está Arca Continental?”

Top-3 (mezcla texto + caption)

[texto] “Arca Continental forma parte de índices como el FTSE4Good... Dow Jones Best-in-Class World Index... Sustainability Yearbook de S&P Global desde 2022...”

[figura] “Página de reporte con logos: S&P Global, Sustainalytics, Dow Jones, MSCI, FTSE4Good, CDP, GRI, Ellen MacArthur Foundation...”

Respuesta del LLM

Arca está en FTSE4Good (Londres), Dow Jones Best-in-Class World Index, Sustainability Yearbook de S&P desde 2022. También figura en MSCI, CDP, GRI y Ellen MacArthur Foundation (*visto en la página de logos*).

Query #2 — recuperación que llega vía una figura

Pregunta

“¿Arca tiene presencia en mercados callejeros en Latinoamérica?”

Top-1 recuperado

[figura] “Dos hombres disfrutando comida en un mercado. Se ven las marcas Inca Kola y Coca-Cola.”

Sin la descripción de la foto, esa información no estaba en el texto plano del PDF. El captioning la rescató.

Gradio — lo muestras al supervisor el martes

```
import gradio as gr

def chat_arca(pregunta):
    return rag(pregunta, k=5)

gr.Interface(
    fn=chat_arca,
    inputs=gr.Textbox(label="Preguntá al Reporte Anual Arca  
↪ 2025"),
    outputs=gr.Textbox(label="Respuesta + cita"),
    examples=["¿En qué índices ESG está Arca?",
              "¿Qué iniciativas tienen para reducir  
↪ emisiones?",
              "¿Cuántos países opera Arca?"],
).launch(share=True)
```

URL pública de Colab. Sin despliegue. *Funciona ya.*

Tres ejercicios para hoy

E1 — aumentar la cobertura

Re-indexa el reporte completo (189 pp). Compará 3 respuestas antes/después de agregar las 80 páginas de financieros.

E2 — tu propio PDF

Subí un PDF tuyo (otro reporte, manual, política), pásalo por `pymupdf4llm`, indexalo. Tres queries que muestren el valor.

E3 — comparar dos librerías

Ejecutá `pymupdf4llm` y luego (opcional) `docling` sobre el mismo PDF. Pegá los outputs lado a lado en una celda. ¿Cuál sirve para tu caso?

🏠 ¿Qué me llevo? — Bloque 6

- ▶ El **asistente Arca v2** responde sobre el reporte real, citando texto y figuras.
- ▶ **La arquitectura no cambió respecto a clase 35.** Solo mejoramos la ingesta (pymupdf4llm + captioning).
- ▶ Lo armas *esta tarde* y lo muestras el lunes.

Un asistente Arca v2 con todo junto. Lo armas hoy.

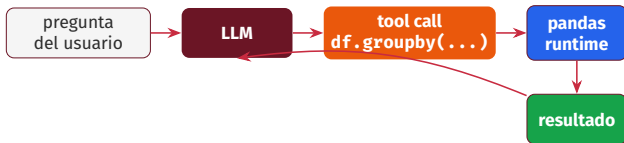
Bonus

¿Y si el dato son números?
RAG sobre tablas para insights

El dato son números, y la tabla es grande

Hasta aquí el RAG *busca párrafos*. Pero a veces la pregunta es “*de los 254 países del dataset OWID CO2, ¿cuáles fueron los 5 mayores emisores en 2022?*”.

Esa tabla tiene **50.411 filas** × **79 columnas**. **No cabe en el contexto del LLM**. Y aunque cupiera, sería desperdicio: el LLM no es bueno para sumar miles de números.



Patrón: el LLM no *mira* la tabla; el LLM *pide al runtime* que la consulte por él. Eso es *tool use*.

El loop completo (el agente lo ejecuta solo)

```
TOOLS = [{"type": "function", "function": {
    "name": "ejecutar_pandas",
    "description": "Ejecuta una expresion pandas sobre el
    ↪ DataFrame df.",
    "parameters": {"type": "object",
        "properties": {"codigo": {"type": "string"}},
        "required": ["codigo"]}}}]

def agent(pregunta):
    msgs = [{"role": "system", "content": SYSTEM},
            {"role": "user", "content": pregunta}]
    for _ in range(4):
        r =
        ↪ client.chat.completions.create(model="llama-3.3-70b-versatile",
        ↪ messages=msgs, tools=TOOLS, tool_choice="auto")
        m = r.choices[0].message
        if not m.tool_calls: return m.content          #
        ↪ respuesta final
        msgs.append(m.model_dump())
        for tc in m.tool_calls:                         #
            ↪ ejecutar tool
            out =
            ↪ eval(json.loads(tc.function.arguments)["codigo"],
            ↪ {"df": df})

            ↪ msgs.append({"role": "tool", "tool_call_id": tc.id, "content": str(out)})
```

El LLM **decide solo** cuándo llamar a la tool y cuándo terminar. **Tú escribes el loop una vez**; el modelo lo orquesta para cada pregunta.

Demo real — OWID CO2 (50.411 filas) preguntada en español

Pregunta

“¿Cuál es el CO2 per cápita de México vs Estados Unidos en 2022?”

Turn 1 — el LLM genera la query

```
df[(df['iso_code'].notna()) & (df['year']==2022) &
(df['country'].isin(['Mexico','United States']))]
[['country','co2_per_capita']]
```

Runtime ejecuta y devuelve

	country	co2_per_capita
29681	Mexico	3.528
48005	United States	14.802

Turn 2 — el LLM redacta el insight

“El CO2 per cápita de México en 2022 fue de **3.528 toneladas**, mientras que el de Estados Unidos fue de **14.802**. Esto muestra que Estados Unidos tiene una emisión per cápita **4.2 veces mayor** que la de México en 2022.”

Cierre

Lo que te llevas

La respuesta a la pregunta del día

En clase 35 armamos un chatbot Arca que cita el Código de Ética. Hoy queremos uno que lea el Reporte Anual Integrado 2025... ¿por qué el RAG simple se rompe aquí, qué le ponemos para que funcione, y cuánto código real escribimos vs cuánto le pedimos a una librería profesional?

**Se rompe porque PyPDF aplana columnas, ignora figuras y destroza tablas.
Lo arreglamos con dos piezas: *pymupdf4llm* (texto + extracción de figuras) y un *modelo multimodal* (Scout) que describe cada figura.
Escribimos ~15 líneas propias; el resto lo hace la librería.
*Producción real = librería pro + multimodal + RAG.***

Gracias

Preguntas?

Clase 36 — RAG multimodal sobre el Reporte Arca 2025

github.com/cmosquerat/arca-diplomado/tree/main/clase-36